

Taxi-Lucky-Index(TLI) for Taxi Information Service (New York)

Spark-based Layered Architecture & Large-scale Data Infrastructure Optimization

---

Team 2

21102042 Seungwoo Baik

21102049 Minhyuk Lee

22102012 Gayoung Lim

22101990 Minseo Kim

Taxi Luck Project

Motivation

We aim to quantify the randomness of taxi arrival times—**"Taxi Luck"**—where duration varies significantly despite identical routes and times.

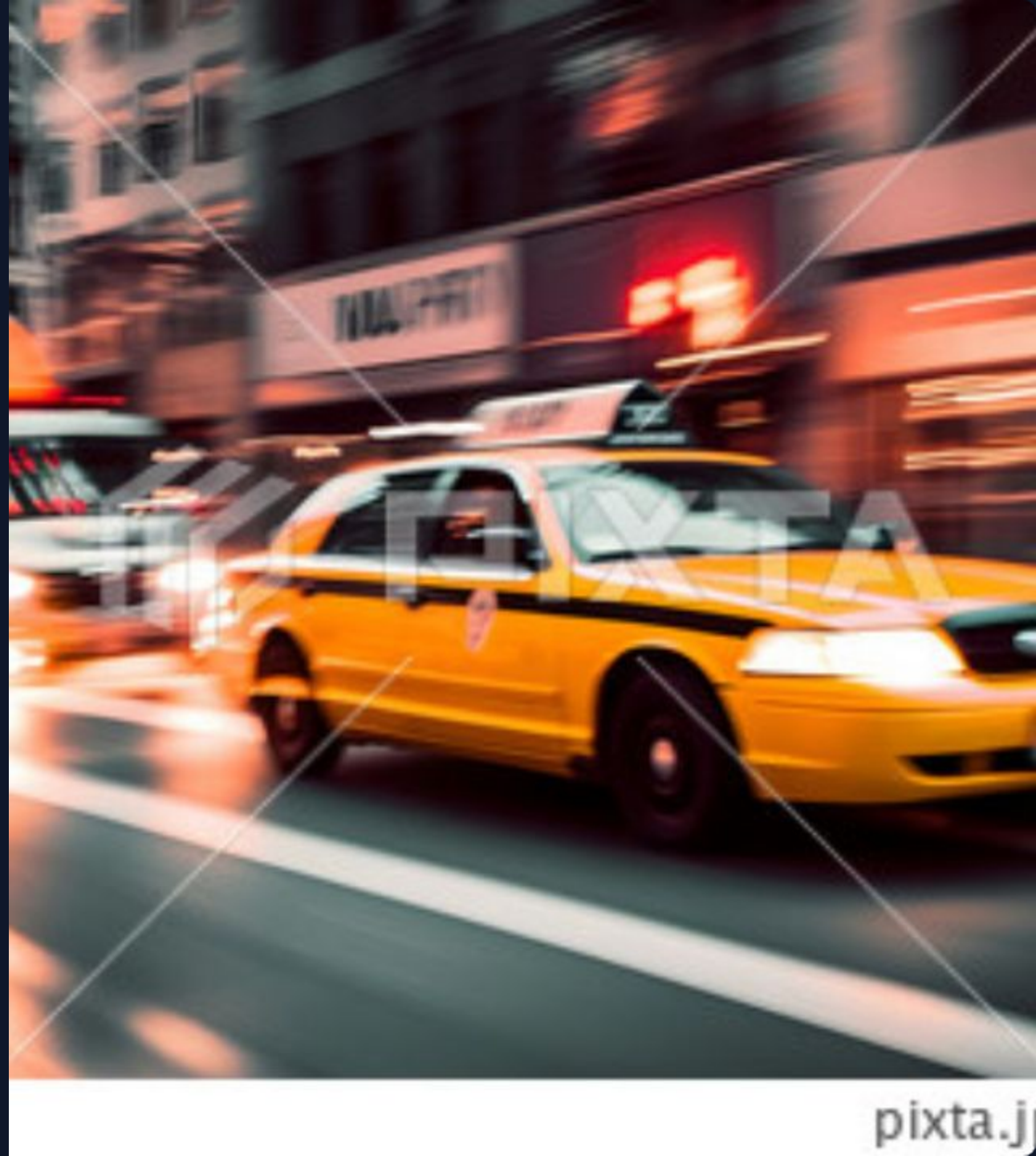
Core Concept

Introducing the **Taxi Luck Index (TLI)**: A novel metric utilizing massive NYC taxi & traffic data to measure travel time variance and quantify unpredictability.

Project Goals

1. Analysis: Calculate TLI by region/time to correlate Traffic Speed Variability with congestion patterns.

2. User Service: Provide a real-time "Taxi Luck Prediction" to visualize ride stability for commuters.



Data Resources



NYC Yellow Taxi Trip Data

Official TLC data providing granular trip details.

- Pickup/Dropoff Datetime
- Trip Distance & Fares
- PULocationID / DOLocationID



NYC Traffic Speed Data

DOT data measuring real-time flow on road segments.

- Average Speed (SPEED)
- Travel Time (TRAVEL_TIME)
- Timestamp (DATA_AS_OF)

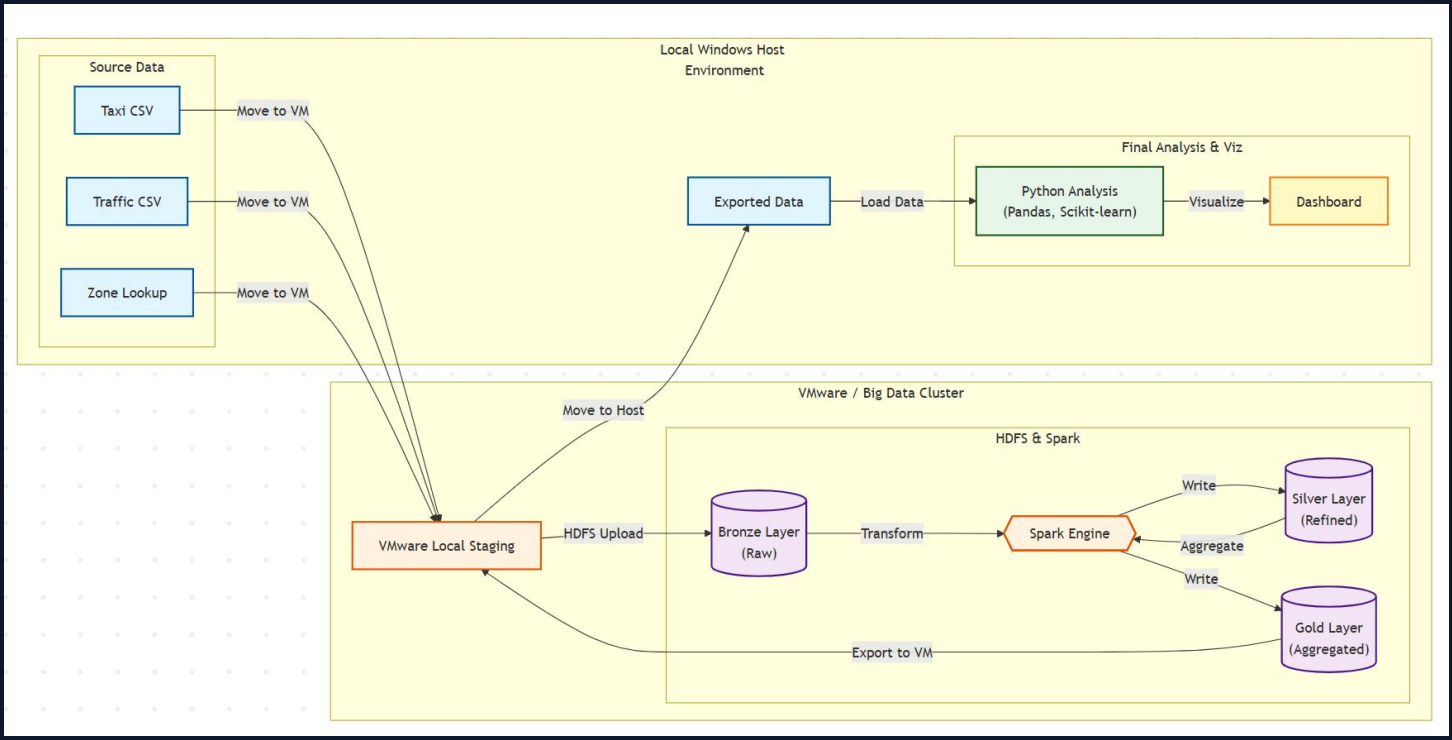


Taxi Zone Lookup

Spatial reference table for geographic mapping.

- LocationID (Primary Key)
- Borough (e.g., Manhattan)
- Zone (Neighborhood Name)

Overall Pipeline Architecture



Data Flow

- **Source:** Local Windows (Taxi CSV, Traffic CSV)
- **Ingestion:** VMware Local → HDFS Upload
- **Processing:** Spark (Bronze → Silver → Gold)
- **Export:** HDFS → VM Local → Windows Local
- **Analysis:** Python (Regression) & Visualization

Final Deliverables: Refined Datasets

TLI_FINAL

PU_Borough	DO_Borough	pickup_ampm	pickup_hour	pickup_min_block	avg_duration	var_duration	TLI	avg_passenger_count	avg_trip_distance	avg_total_amount
Manhattan	Bronx	AM	6	0	22	568	0.44	1.256317689530690	9.26	33.96
Queens	Staten Island	PM	9	15	43	558	0.22	1.3137254901960800	26.6	89.53
Staten Island	Brooklyn	AM	1	15	23	95	0.07	1.0	5.29	49.15
Bronx	Staten Island	PM	4	45	114	1398	0.2	1.0	10.4	95.85
Brooklyn	Queens	AM	7	45	34	1181	0.58	1.2872340425531900	315.09	48.96
Bronx	Queens	AM	1	0	22	547	0.42	1.8333333333333300	9.7	45.35
Manhattan	Bronx	AM	5	45	21	492	0.4	1.2591792656587500	8.92	32.86
Staten Island	Staten Island	PM	12	45	31	2552	1.39	1.3846153846153800	1.53	31.5
Brooklyn	Manhattan	PM	3	45	30	1148	0.64	1.4775132275132300	134.24	31.39
Bronx	Staten Island	PM	5	0	103	1126	0.18	1.0	30.12	100.6
Queens	Queens	AM	5	15	14	1170	1.41	1.433093994778070	6.31	36.28
Brooklyn	Manhattan	PM	4	0	30	964	0.54	1.467403314917130	5.33	33.55
Brooklyn	Queens	AM	8	0	34	874	0.43	1.2244897959183700	11.37	47.64
Brooklyn	Manhattan	AM	1	30	21	541	0.43	1.455958549222800	5.58	30.11
Queens	EWR	PM	9	30	52	532	0.17	1.6428571428571400	33.01	137.57
Manhattan	Staten Island	PM	7	15	47	751	0.27	1.2962962962963000	18.23	83.7
Staten Island	Staten Island	AM	10	30	66	3185	0.8	1.0	0.2	106.36
Brooklyn	Staten Island	AM	9	30	45	1198	0.44	1.0	21.13	69.65
Staten Island	Bronx	AM	7	0	47	389	0.14	1.0	27.22	76.51

CI_FINAL

BOROUGH	tf_ampm	tf_hour	tf_min_block	avg_speed	CI
Staten Island	AM	10	15	37.94	0.24
Brooklyn	PM	7	30	34.8	0.3
Queens	PM	12	15	38.02	0.24
Bronx	PM	9	15	36.73	0.27
Staten Island	AM	4	15	42.65	0.15
Staten Island	AM	11	0	37.61	0.25
Queens	PM	6	15	32.6	0.35
Staten island	AM	1	15	40.43	0.19
Staten Island	AM	5	0	43.17	0.14
Queens	PM	7	0	35.41	0.29
Staten island	AM	2	0	40.57	0.19
Staten island	AM	10	45	37.09	0.26
Staten Island	AM	7	45	38.02	0.24
Staten Island	PM	2	45	35.36	0.29
Queens	PM	9	45	41.93	0.16
Staten island	AM	11	30	36.8	0.26
Manhattan	AM	1	0	22.8	0.54
Staten Island	AM	8	30	37.79	0.24
Staten island	AM	4	45	40.98	0.18

Analysis Ready Mart

Processed approx. 30GB of Raw Data into a final Mart (CSV) ready for analysis.

TLI Data:
Includes Uncertainty metrics by PU/DO Borough & Time_Block.

CI Data:
Includes Congestion Index by Borough, Hour & Speed.

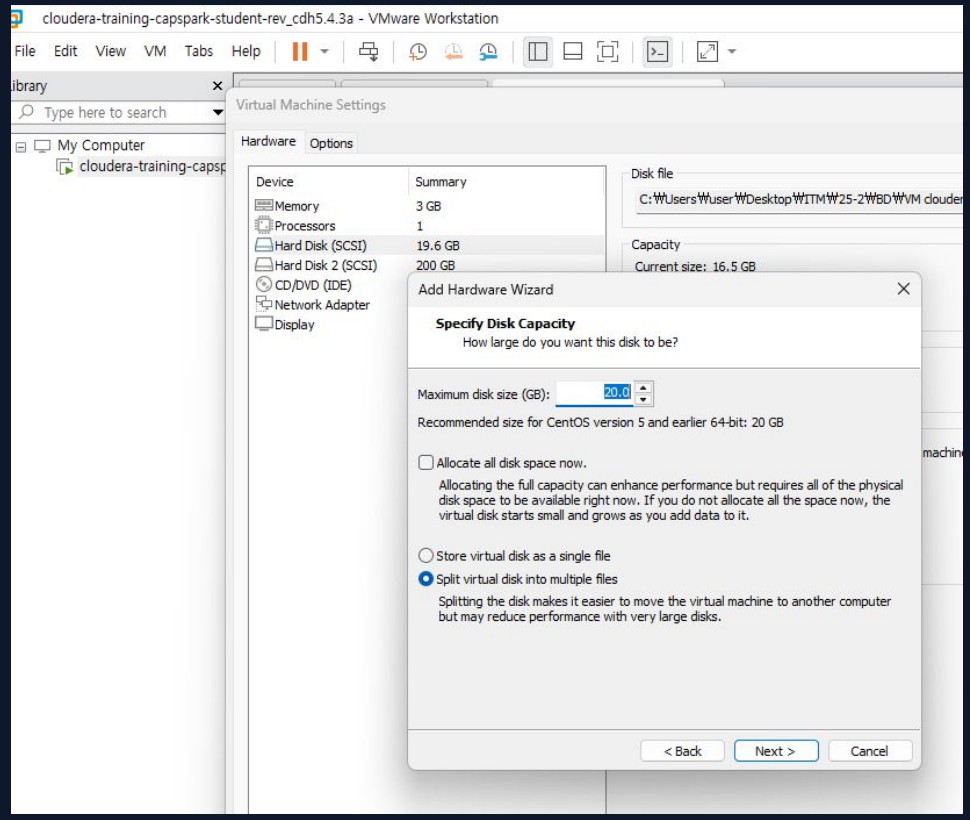
[Troubleshooting] Storage Optimization

Problem

Initial VM capacity (20GB) was insufficient for 30GB Raw Data, causing upload failures & Permission denied errors.

Solution (Debugging)

- **Disk Expansion:** Expanded VMware Disk from 20GB to 200GB.
- **Partitioning:** Mounted and formatted Linux partition (sdb).
- **Config:** Reconfigured HDFS data directory paths.




```
[training@localhost ~]$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sr0     11:0    1 1024M  0 rom
sda      8:0    0 19.6G  0 disk
├─sda1   8:1    0 17.6G  0 part /
└─sda2   8:2    0    2G  0 part [SWAP]
sdb      8:16   0  200G  0 disk
└─sdb1   8:17   0  200G  0 part
```

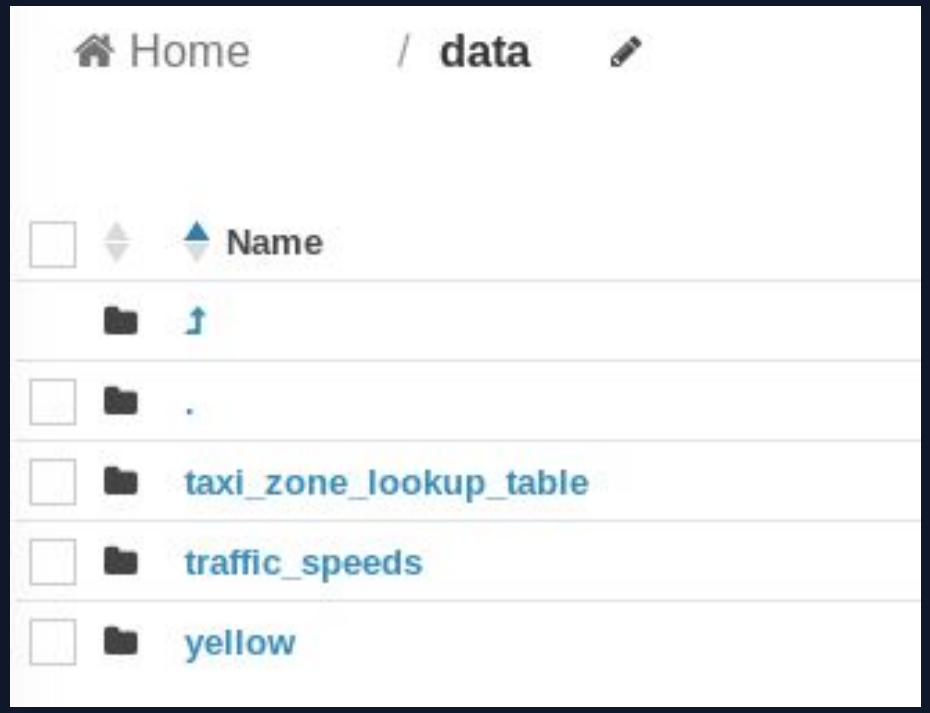
Data Ingestion & Automation Plan

ex) Look up table

```
C:\Users\user\Desktop\ITM\25-2\BD\datasets>scp -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedKeyTypes=+ssh-rsa ".\taxi
_zone_lookup.csv" training@192.168.132.128:/home/training/
training@192.168.132.128's password:
taxi_zone_lookup.csv                                          100%   12KB   2.0MB/s   00:00
```

```
[training@localhost ~]$ hdfs dfs -put /home/training/taxi_zone_lookup.csv /data/
yellow/
```

upload all data to hdfs



Manual Ingestion Process

- **Transfer:** `scp` from Local Windows to VM Linux.
- **Unzip:** Decompress inside VM for efficiency.
- **HDFS Load:** `hdfs dfs -put` to /data/ directory.

Future Work: Automation

Current static data uses manual batch processing.
For real-time pipelines, we plan to introduce **Kafka & Airflow**.

Data Processing Strategy (Layered Architecture)

Adopted Spark-based Layered Architecture for recovery and traceability.

Bronze Layer

Preserve Raw Data (Parquet).

Managed under /data/ directory.

Silver Layer

Refined & Filtered Data.

Revertible processing steps.

Gold Layer

Final Aggregation.

Spark SQL Metadata Management.

* All layers use **Parquet** format for compression efficiency and speed.

[Bronze Layer] Raw Data Loading

Schema Inference

Utilized Spark's inferSchema option to automatically detect data types from large CSV files.

```
from pyspark.sql import SQLContext
sqlc = SQLContext(sc)
taxi_path =
"hdfs://localhost:8020/data/yellow/2022_Yellow_Taxi_Trip_Data.csv"

df_taxi_raw = sqlc.load(
    source="com.databricks.spark.csv",
    path=taxi_path,
    header="true",
    inferSchema="true"
)

df_taxi_raw.printSchema()
df_taxi_raw.show(5, False)
```

Ref: PDF Page 15

[Silver Layer] Data Quality

```
bronze_taxi_path = "hdfs:///data/bronze_layer/yellow"
df_taxi = sqlc.load(bronze_taxi_path, "parquet")
df_taxi.registerTempTable("taxi_raw")

df_taxi_clean = sqlc.sql(
    "SELECT * "
    "FROM taxi_raw "
    "WHERE tpep_pickup_datetime IS NOT NULL "
    "  AND tpep_dropoff_datetime IS NOT NULL"
)
df_taxi_clean.registerTempTable("taxi_clean")
```

Ensuring Completeness

Applied rigorous exception handling logic.

- **Null Handling:**
  Removed records missing Pickup/Dropoff times.
  WHERE ... IS NOT NULL

[Silver Layer] Feature Engineering

Time Blocking & Columns

Creating derived features for time-series analysis.

- **15-min Blocking:** Categorized into 00, 15, 30, 45 using CASE WHEN.
- **Duration Calculation:** Dropoff Time - Pickup Time (Seconds).
- **AM/PM:** Created a distinction column for day/night patterns.

```
df_minute_block = sqlc.sql(
    "SELECT *, "
    "  CASE "
    "    WHEN CAST(SUBSTRING(tpep_pickup_datetime, 15, 2) AS INT)
BETWEEN 0 AND 14 THEN '00' "
    "    WHEN CAST(SUBSTRING(tpep_pickup_datetime, 15, 2) AS INT)
BETWEEN 15 AND 29 THEN '15' "
    "    WHEN CAST(SUBSTRING(tpep_pickup_datetime, 15, 2) AS INT)
BETWEEN 30 AND 44 THEN '30' "
    "    ELSE '45' "
    "  END AS pickup_min_block "
    "FROM taxi_clean"
)
df_minute_block.registerTempTable("taxi_minute_block")
```

```
    "  CASE "
    "    WHEN (d_h * 3600 + d_m * 60 + d_s) >= (p_h * 3600 + p_m *
60 + p_s) "
    "      THEN (d_h * 3600 + d_m * 60 + d_s) - (p_h * 3600 + p_m
* 60 + p_s) "
    "    ELSE "
    "      (d_h * 3600 + d_m * 60 + d_s) - (p_h * 3600 + p_m * 60
+ p_s) + 86400 "
    "  END AS duration_seconds "
    "FROM taxi_15m_hms"
)
```

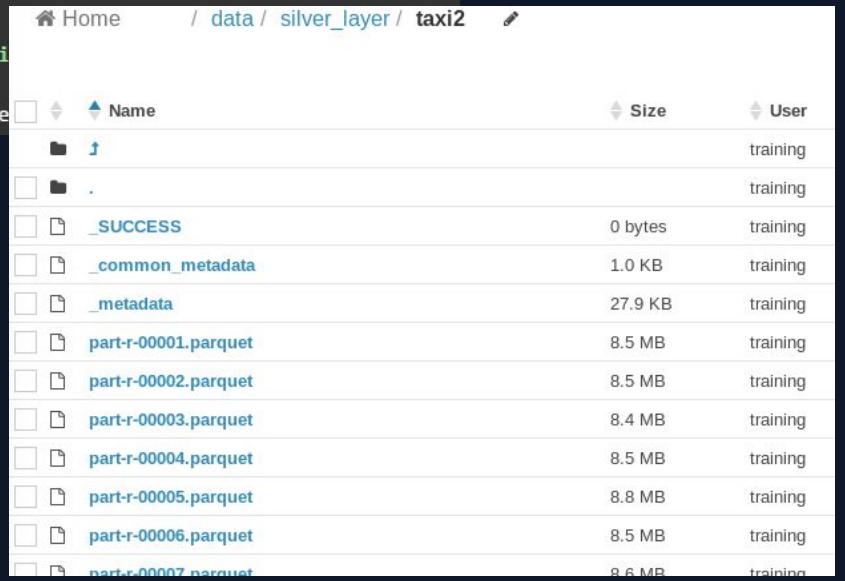
[Silver Layer] Data Optimization

```
df_silver.registerTempTable("taxi_silver")

df_taxi_ampm = sqlc.sql(
    "SELECT "
    "  passenger_count, "
    "  trip_distance, "
    "  total_amount, "
    "  PULocationID, "
    "  DOLocationID, "
    "  pickup_min_block, "
    "  pickup_date, "
    "  pickup_hour, "
    "  duration_seconds, "
    "  SUBSTRING(tpep_pickup_datetime, 21, 2) AS pickup_ampm "
    "FROM taxi_silver"
)

silver_path = "hdfs:///data/si

df_taxi_ampm.saveAsParquetFile
```



Column Pruning

Lightweighting dataset for high-performance processing.

- **Action:** Removed detailed fare columns (tax, tips, etc.) unnecessary for TLI analysis.
- **Outlier Handling:**
  Removed Negative Duration (Dropoff < Pickup).
  if dur < 0: return None
- **Result:** Significantly improved processing speed by retaining only core columns (Location, Time, Speed).

[Gold Layer] Data Integration

Dimension Mapping

Joining Taxi Data with Zone Lookup Table.

```
"LEFT JOIN taxi_zone zp ON t.PULocationID = zp.LocationID "
"LEFT JOIN taxi_zone zd ON t.DOLocationID = zd.LocationID "
```

Logic: Converted ID-based location data into human-readable **Borough names** (e.g., Manhattan, Queens).

```
df_taxi_borough = sqlc.sql(
    "SELECT "
    "  t.passenger_count, "
    "  t.trip_distance, "
    "  t.total_amount, "
    "  t.PULocationID, "
    "  zp.Borough AS PU_Borough, "
    "  t.DOLocationID, "
    "  zd.Borough AS DO_Borough, "
    "  t.pickup_min_block, "
    "  t.pickup_date, "
    "  t.pickup_hour, "
    "  t.duration_seconds, "
    "  t.pickup_ampm "
    "FROM taxi_silver t "
    "LEFT JOIN taxi_zone zp ON t.PULocationID = zp.LocationID "
    "LEFT JOIN taxi_zone zd ON t.DOLocationID = zd.LocationID "
)
```

[Platinum Layer] TLI Calculation & CI Calculation

Taxi Luck Index (Uncertainty Metric)

Quantifying the irregularity of travel times.

$$TLI = \frac{STDDEV\ (\ Duration\ )}{AVG\ (\ Duration\ )}$$

Grouped by (Borough, Hour, Block) and calculated using Spark SQL aggregation functions.

Congestion Index

Measuring road traffic congestion levels.

$$CI = 1 - \frac{Average\ Speed}{50mph}$$

Calculated based on free-flow speed (50mph). Closer to 1 indicates higher congestion.

Data Export & Integration

Export Process Flow

- **HDFS → VM:** hdfs dfs -get (Platinum Layer)
- **VM → Windows:** scp transfer
- **Merge:** Python (Pandas) to merge distributed Parquet files into single CSVs.

Result: CI_FIANL.csv, TLI_FINAL.csv

```
[training@localhost ~]$ sudo mkdir -p /data1/ci_local
[training@localhost ~]$ sudo chown -R training:training /data1/ci_local
[training@localhost ~]$ hdfs dfs -get /data/platinum_layer/ci/* /data1/ci_local/
```

```
PS C:\Users\user\Desktop\ITM\25-2\BD\datasets\CI> scp -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedKeyTypes=+ssh-rsa `
>>   training@192.168.132.132:/data1/ci_local/* .
training@192.168.132.132's password:
_common_metadata                                      100%  614   199.9KB/s   00:00
_metadata                                             100%   86KB  14.0MB/s   00:00
part-r-00001.parquet                                  100% 1526   372.6KB/s   00:00
part-r-00002.parquet                                  100% 1529   373.3KB/s   00:00
part-r-00003.parquet                                  100% 1495   365.0KB/s   00:00
part-r-00004.parquet                                  100% 1367   445.0KB/s   00:00
part-r-00005.parquet                                  100%  614   299.8KB/s   00:00
part-r-00006.parquet                                  100% 1550   504.6KB/s   00:00
part-r-00007.parquet                                  100% 1591   310.7KB/s   00:00
part-r-00008.parquet                                  100% 1367   667.5KB/s   00:00
```

```python
import pandas as pd
import glob
import os

parquet_folder = r"C:\Users\user\Desktop\ITM\25-2\BD\datasets\CI"

# Read all parquet files within that path
files = glob.glob(os.path.join(parquet_folder, "*.parquet"))

print("Number of Parquet files found:", len(files))
if len(files) == 0:
    raise FileNotFoundError(" No parquet files found in the folder!")

# Concatenate multiple parquet files into a single DataFrame
df_list = []
for f in files:
    print("Reading:", f)
    df_list.append(pd.read_parquet(f))

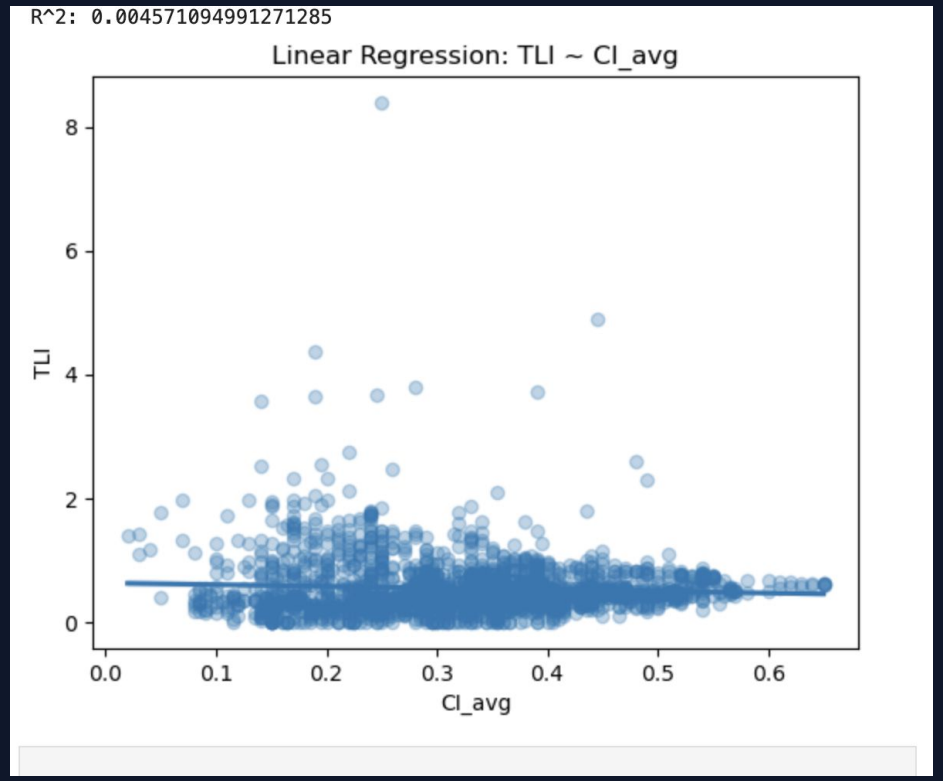
df = pd.concat(df_list, ignore_index=True)

output_csv = os.path.join(parquet_folder, "ci_final.csv")

df.to_csv(output_csv, index=False, encoding="utf-8-sig")

print("Complete! CSV file saved at →", output_csv)
```

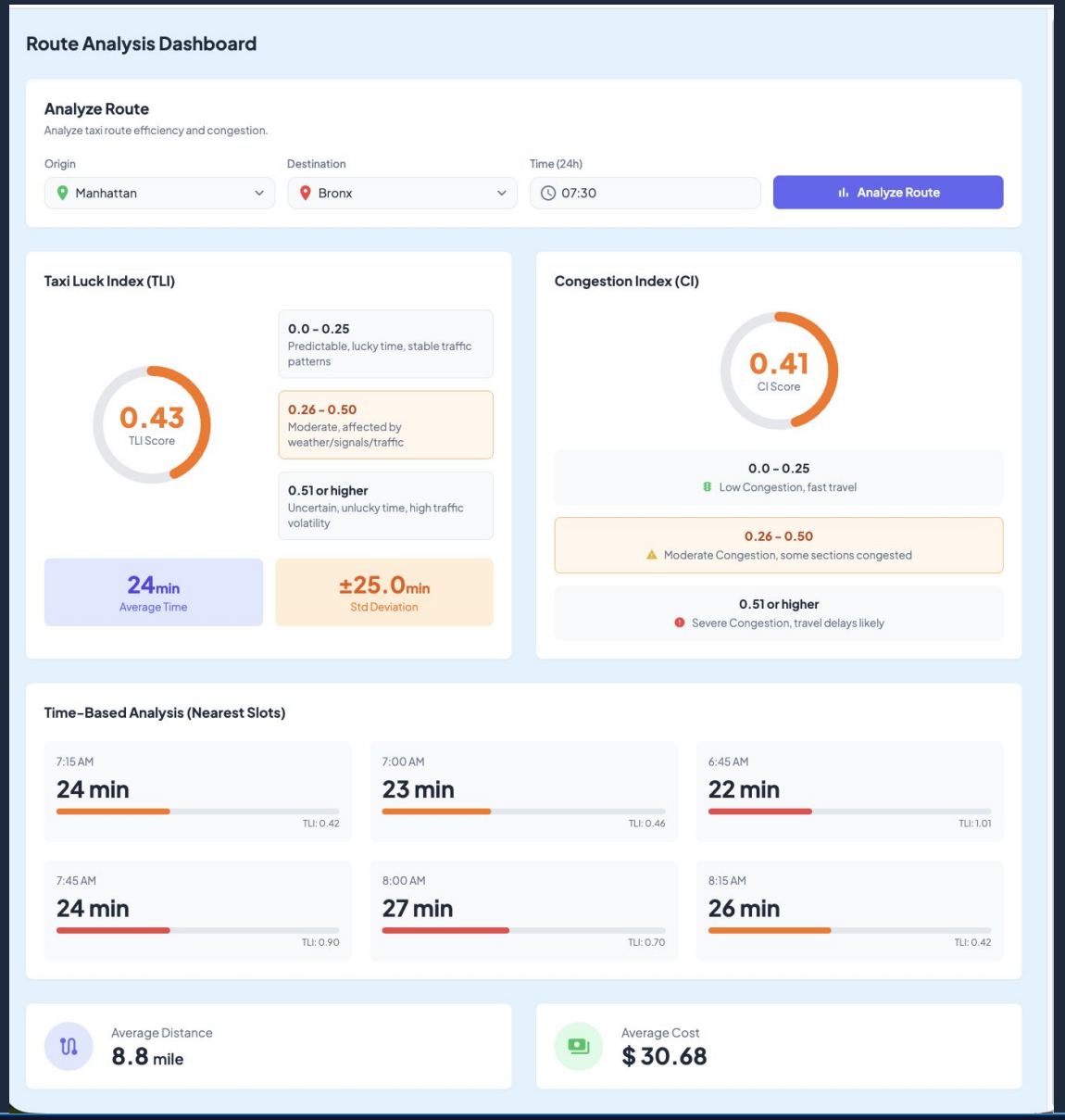
Regression Analysis Results



Insight

Linear Regression between CI (Congestion) and TLI (Uncertainty).

- **Result:** Low R^2 value (~0.004).
- **Interpretation:** The linear relationship is weak due to the broad resolution of 'Borough' data.
- **Conclusion:** TLI captures different characteristics than simple congestion.



Final Result: Route Analysis Dashboard

Conclusion & Key Takeaways

Engineering Achievements

- **Infrastructure:** Solved 'Permission Denied' & storage issues (20GB → 200GB).
- **Architecture:** Implemented Spark-based **Layered Architecture** (Bronze/Silver/Gold) for recovery.
- **Performance:** Optimized processing speed via Parquet format & Column Pruning.

Future Improvements

- **Automation:** Upgrade manual batch pipeline to Real-time Automation using **Kafka & Airflow**.
- **Analysis:** Acquire Zone/Road-level data to resolve the low resolution issue of 'Borough' based analysis.